Quaestio implementation and design decisions

Completed October 7 2026. Package version 0.1.0. Prepared for Srivan Nagabhyru.

1 Outcome and scope

Quaestio now has a working MVP and several practical extensions. A user can install the package, stream a text or NDJSON log through validated events, calculate severity counts and the ERROR rate, rank repeated errors, and obtain a terminal, JSON, or CSV report. The implemented command is `quaestio analyze`; `python -m quaestio` is an alternative entry point.

The existing normalized architecture was preserved: input adapter -> LogEntry -> optional selection -> analyzer -> AnalysisResult -> report renderer -> CLI. The parser and analyzer do not depend on Typer or Rich. No database, web server, AI runtime, or new runtime dependency was introduced.

All 58 tests passed both in the development environment and against an installed wheel in a clean virtual environment outside the repository. The sample reports four entries, two INFO, one WARNING, one ERROR, and a 0.25 error rate. A CI matrix is configured for Python 3.11-3.14, but remote CI has not been run during this work.

This milestone completes the local software MVP. Publishing a release and committing or pushing the working tree were not performed. External GitHub and Notion state was not verified or modified. Their existence in the original handoff is historical context, not validation evidence for this implementation.

2 Starting point and changes made

The complete Claude handoff was reviewed together with the repository. The working models, text parser, generator-based file reading, analyzer counters, repeated-error ranking, and 11 existing tests formed the baseline. The analyzer and its tests already contained uncommitted ranking work; that work was preserved. The CLI file existed but was empty, causing the installed command to fail when importing its missing app. README was empty. The sample log existed locally but was excluded by the generic `*.log` ignore rule. CHANGELOG and CONTRIBUTING were absent; docs and scripts had no substantive content. The actual packaging author name was already correct.

Added files and capabilities

- `src/quaestio/cli.py`: implemented the previously empty Typer application and analyze command, including stdin, options, output saving, diagnostics, and exit codes.
- `src/quaestio/reporting.py`: added Rich text rendering and JSON/CSV serialization.
- `src/quaestio/__main__.py`: enabled module-based invocation.
- `examples/sample.ndjson`: added a structured equivalent of the text fixture.

- `tests/conftest.py`, `tests/test_cli.py`, and `tests/test_streaming.py`: added portable fixture resolution, command tests, file-failure tests, and streaming regressions.
- `scripts/benchmark.py`: added reproducible synthetic parser/analyzer measurements.
- `.github/workflows/tests.yml`: configured tests, wheel building, and installed-command smoke checks for Python 3.11, 3.12, 3.13, and 3.14 on Linux.
- `MANIFEST.in`: declared examples, tests, development guidance, benchmarks, and Markdown documentation for source distributions. Wheels contain the runnable package.
- README, CHANGELOG, CONTRIBUTING, architecture notes, benchmark observations, and this report: added installation, contracts, reasoning, evidence, and future-work guidance.

Modified existing functionality

The text parser still understands DATE TIME LEVEL SOURCE MESSAGE and produces the same LogEntry model. It now reports file/line locations, ignores blank physical lines, accepts file BOMs, and optionally skips malformed records through a callback. It also provides parse_lines and parse_json_line adapters. Existing parse_file(path) calls continue to work. LogParseError now subclasses ValueError; existing catches of LogParseError still work, while callers catching ValueError will also catch it.

The analyzer retains its single-pass counting and ERROR-only rate. It adds type annotations, a positive configurable top limit, heap-based ranking, and a reusable lazy source/severity selector. Default top remains five. Equal-frequency messages retain first appearance order. Domain model field names and AnalysisResult schema were preserved; LogEntry formatting and documentation were clarified.

The Git ignore rule now permits synthetic logs under examples. Two existing tests resolve fixtures through a shared absolute-path fixture instead of depending on the shell's working directory. The existing tests were extended rather than replaced. No functional component, model field, dependency, or existing test was removed. The full-sort ranking expression was replaced by equivalent top-k selection.

3 MVP completion

- Validated LogLevel, LogEntry, and AnalysisResult: complete.
- Text parser and project-specific parse errors: complete.
- Streaming file processing: complete, with explicit memory limits documented below.
- Severity counting, empty-input handling, ERROR rate, repeated errors, top five: complete.
- Thin CLI using Typer and Rich: complete and tested through the installed command.
- Sample input, package entry point, tests, installation and usage README: complete locally.
- Version control structure: present; synthetic fixtures are now eligible for tracking. The implementation is still an uncommitted working-tree milestone.
- Public release or updated GitHub publication: not performed; this is a delivery activity, not a missing local analysis feature.

There are no partially implemented local analysis MVP features. Beyond the MVP, the implementation adds NDJSON, stdin, JSON/CSV export, file saving, exact filters, configurable top

errors, tolerant parsing, bounded diagnostics, BOM handling, portable fixtures, packaging checks, benchmarks, and CI configuration. Benchmarks are executed; remote CI is configured but unexecuted.

4 How the implementation works

Reading and normalization

parse_file opens a UTF-8 file lazily, using utf-8-sig so a leading file BOM is consumed. It delegates iteration to parse_lines. parse_lines accepts any iterable of strings, ignores blanks, selects the explicitly requested parser, and yields one validated LogEntry at a time. A malformed record becomes a LogParseError with source and physical line number. The exception chain retains its original cause for debugging.

Text parsing keeps split(maxsplit=4), so messages may contain spaces. Timestamps use YYYY-MM-DD HH:MM:SS and have no timezone. Levels remain uppercase. A text source is a single token. NDJSON parsing delegates one complete JSON object per physical line to Pydantic's LogEntry.model_validate_json. The same required fields and enum validation apply. ISO timestamp strings may include offsets; existing Pydantic timestamp coercion also applies. Extra JSON keys are ignored, as in the existing model configuration.

Streaming is preserved end to end: the CLI passes the parser generator through the selector to the analyzer. It explicitly closes the parser generator in a finally block. Python API callers who stop reading a file generator early must close it themselves or use contextlib.closing. Fully exhausted or failed generators leave the file context.

Selection and aggregation

select_entries returns a generator expression matching an exact source and/or exact LogLevel. It does not buffer or mutate events. The analyzer consumes only selected entries, so counts and the rate denominator refer to that selected population. Malformed records are parsed before filtering and still fail strict mode; selecting one source cannot conceal invalid records elsewhere in the stream.

The analyzer increments exactly one severity counter per event. A dictionary retains counts for distinct ERROR messages. ERROR rate is error_count / total_entries, with zero for empty input. CRITICAL has its own counter but does not contribute to this rate or top errors. Messages are grouped exactly, across sources, without case folding, parameter stripping, or fuzzy similarity. heapq.nlargest chooses the top k after counting.

Presentation and saving

AnalysisResult stays numeric and independent of presentation. JSON uses Pydantic serialization and represents tuple pairs as JSON arrays. CSV uses Python's csv writer with section,name,value columns; summary rows precede top-error rows. This handles commas, quotes, and Unicode without manual escaping. Rich produces a human-readable report, using Text for raw messages so bracketed log content is not parsed as markup. The text renderer uses a fixed 100-column layout and wraps long messages.

The CLI emits reports to stdout and diagnostics to stderr. With --output-file, it renders first, writes a temporary UTF-8 file in the destination directory, closes it, and replaces the destination with os.replace. A failure removes the temporary file. Input equality checks include resolved

paths and samefile checks for existing hardlinks. There is no automatic creation of missing output directories.

Command examples

```
quaestio analyze examples/sample.log
quaestio analyze examples/sample.ndjson --format ndjson --output json
cat examples/sample.log | quaestio analyze - --output json
quaestio analyze server.log --skip-invalid --source database --top 10
quaestio analyze server.log --output csv --output-file report.csv
```

Input format is --format text|ndjson. Output format is --output text|json|csv. --level is exact matching, not a severity threshold. --top accepts 1-1000 in the CLI; the core accepts any positive integer. Exit code 0 means successful analysis, including explicit skip mode; 1 means an input/output failure; 2 means invalid command usage. An ERROR event is data, not a command failure. Empty input yields a valid zero report.

5 Design decisions and alternatives

D1 Preserve the functional core and add small adapters

Decision: extend existing functions rather than introduce a service framework or rewrite models. Parser, analyzer, selector, and report renderer have distinct responsibilities. The CLI coordinates them and manages command-specific files and error presentation.

Why and benefit: the handoff already established a useful normalized model and passing core. Keeping it preserves prior work, limits regression risk, and makes a future API able to call the same engine. Keyword-only options extend old calls without changing their defaults. Typing and docstrings clarify contracts without adding runtime machinery.

Trade-off: there is no dependency injection container, parser registry, abstract class, or central application service. Some coordination remains in the command function.

Alternatives: a monolithic CLI is initially shorter but couples analysis to terminal behavior and complicates reuse. An object hierarchy or service container can help when many engines or lifecycle-managed resources exist, but introduces indirection before that need is present. Small adapters provide the necessary separation at lower cost.

D2 Explicit NDJSON alongside the existing text format

Decision: retain text as the default and add NDJSON selected by --format. Reuse Pydantic for validation; ignore extra keys rather than expand the model prematurely.

Why and benefit: NDJSON is naturally one-record-per-line and streams without loading an entire JSON document. Structured applications can now use Quaestio without converting their records into text. Shared validation keeps parser outputs consistent.

Trade-off: field names must match timestamp, level, source, and message. Extra metadata is discarded. Multi-line objects, JSON arrays, stack traces, and syslog are unsupported. Levels are case sensitive. Pydantic coercion is convenient but not a strict ISO-only contract.

Alternatives: whole-file json.load is simpler for arrays but violates the streaming goal. Automatic detection is convenient but can misclassify ambiguous input and complicate stdin and failure handling. A configurable mapping layer would support diverse schemas but needs concrete

examples and a stable mapping contract. Syslog first would broaden support but brings timestamp, hostname, and dialect ambiguity. Explicit NDJSON delivers structured input with a small, testable contract.

D3 Strict parsing by default with explicit tolerant mode

Decision: fail fast on malformed nonblank records unless --skip-invalid is provided. Blank lines are ignored. Skip mode exposes errors through a callback and reports a count; only the first three individual warnings are printed.

Why and benefit: an incomplete report should not silently look complete. File/line locations make repair practical. Optional tolerance makes imperfect logs usable without forcing users to pre-clean them. Bounded warnings prevent stderr output proportional to millions of bad records while retaining the exact skipped count.

Trade-off: strict mode emits no partial report. Tolerant results exclude rejected records and can be biased; all-invalid input succeeds with a zero report and warnings. Skipped counts remain diagnostics rather than AnalysisResult fields, so scripts that need completeness must use strict mode or inspect stderr. Raw records are not repeated in most diagnostics, reducing accidental disclosure but requiring users to open the line.

Alternatives: silently skipping is convenient but undermines trust. Always failing prevents analysis of damaged data. Returning a list of errors retains too much memory. A structured run-result envelope with parse statistics would aid automation but changes the public JSON schema and expands the domain contract. The callback is sufficient now; a versioned report envelope is a sensible future enhancement.

D4 Keep existing severity and grouping semantics

Decision: preserve ERROR-only rate and exact message grouping; show CRITICAL separately. No model schema change is needed.

Why and benefit: previously passing tests and handoff behavior remain valid. Users can compare results across the milestone without a silent metric change. The text label and README explicitly describe the numerator.

Trade-off: a CRITICAL-only log has a zero ERROR rate. Different IDs or parameters make otherwise similar messages distinct. Identical messages from different sources merge.

Alternatives: combining ERROR and CRITICAL into a failure rate is often operationally useful, but should be a separately named metric. Severity-weighted scoring imposes arbitrary weights. Fuzzy normalization can collapse meaningful distinctions. Source-plus- message grouping gives attribution but changes existing ranking. Preserve the metric now; add explicitly named alternatives if a real investigation requires them.

D5 Retain exact counting and use standard-library top-k selection

Decision: keep the dictionary of exact message counts and use heapq.nlargest instead of sorting every message when only a small top list is needed. Ties keep first appearance order.

Why and benefit: the output remains equivalent to sorted(..., reverse=True)[:k], as specified by Python's heapq documentation. Ranking needs roughly O(U log k) work for small k rather than O(U log U) sorting, and avoids a full-size temporary ranked list. A local ranking comparison corroborated the benefit; details appear in the benchmark section.

Trade-off: exact counting still requires O(U) dictionary memory for U distinct ERROR messages. Very large k may approach full-sort cost. This does not make memory independent of the number of unique errors or the length of individual records.

Alternatives: retaining the full sort is clear but allocates and sorts unnecessary results. Counter.most_common is idiomatic and viable, but replacing the counting code would add churn for similar top-k behavior. Approximate heavy-hitter algorithms bound memory but can misstate counts. SQLite or disk-backed aggregation is exact but introduces I/O, schema, and cleanup concerns. Async or multiprocessing adds coordination without measured need. Exact dictionary counts plus a library function suit this toolkit.

D6 Reusable exact filters applied before aggregation

Decision: use a lazy core selector for source and severity. The analyzer summarizes the selected population; the CLI does not calculate business metrics.

Why and benefit: users can investigate one component or severity without an external preprocessing step. Streaming and core reuse remain intact. A no-match filter naturally returns the existing empty result.

Trade-off: filtering ERROR yields a 100% rate when any selected events exist. The rate is not the rate of the original whole file. Exact matching does not offer regexes, minimum severity, time ranges, or combinations of multiple sources.

Alternatives: filtering only the finished report cannot produce correct subset counts. CLI-only filter logic obstructs reuse. A general query language offers flexibility but requires syntax, error handling, and semantics well beyond the current need. Two exact filters are useful and easy to explain; richer filtering should be motivated by usage.

D7 Separate input format, output format, and output destination

Decision: --format chooses parsing, --output chooses report representation, and --output-file chooses the destination. Renderers only consume AnalysisResult.

Why and benefit: flags cannot ambiguously mean JSON input versus JSON output. Scripts can parse stdout without banners or warnings. JSON preserves the domain schema and ratios; CSV provides a simple tabular export; text remains readable for humans.

Trade-off: CSV is a tagged row schema rather than one very wide record. Top errors are variable-length rows. CSV messages are preserved rather than spreadsheet-sanitized; formula-like strings must be treated as untrusted when imported into spreadsheet apps. Text output wraps at 100 columns and has no interactive color behavior.

Alternatives: mixing serialization into the analyzer couples data to presentation. A nested report envelope is richer but changes the contract. Two CSV files separate summary and errors cleanly but complicate CLI use and stdout. A wide CSV row is awkward for changing top limits. Automatically altering formula-like messages improves some spreadsheet workflows but loses exact log text. The tagged schema and raw preservation are appropriate for a general analysis export, with the import caveat documented.

D8 Atomic file replacement and input protection

Decision: save reports through a temporary file in the output directory, then os.replace. Reject resolved path equality and existing samefile aliases before analyzing.

Why and benefit: malformed input never truncates an old report, and write/replace failures leave it intact. The same-directory temporary file avoids cross-filesystem replacement. Hardlink and symlink checks prevent obvious input destruction. Failure tests verify the old report remains and temporary files are cleaned.

Trade-off: replacing a file changes its inode and usually its permissions to the temporary file's restrictive defaults. Existing metadata is not preserved. A symlink output to a separate report is replaced as a path entry, not followed for the final write. No fsync or concurrent-path lock is used; atomic replacement is not power-loss durability or a complete defense against a concurrent path race. Output directories must already exist.

Alternatives: direct writes are shorter but may truncate on failure. Append mode avoids replacement but mixes separate reports. Refusing every existing output protects all files but frustrates repeatable automation. Locks and directory descriptors address concurrency attacks but add platform-specific complexity. Transactional storage is unnecessary for a local file report. Atomic replacement is proportionate here.

D9 Test behaviors and verify the installed distribution

Decision: retain existing tests, add command/streaming/file-failure regressions, and run the suite against both the working tree and a wheel installed into a clean environment. Fixtures resolve relative to their test location rather than the current directory.

Why and benefit: source imports can mask missing package files or a broken entry point. A clean installation outside the checkout verifies the deliverable users actually run. One-pass streams and file-close tests exercise resource contracts. Failure injection validates output preservation without unreliable OS permission tricks.

Trade-off: some tests use monkeypatch to simulate write and permission failures. These check application behavior, not every filesystem implementation. The suite does not measure coverage percentages or certify all possible input combinations.

Alternatives: manual smoke checks alone miss regressions. Snapshotting every Rich output character overconstrains cosmetic formatting; assertions target counts, messages, schemas, and exit behavior instead. A large fuzzing/property-testing dependency can find more edge cases, but is deferred until the format contract expands. Current pytest and Typer's CliRunner provide substantial confidence without a new test framework.

D10 Measure separately and add CI without adding a runtime service

Decision: use a standard-library benchmark and a Linux Python-version CI matrix. The benchmark generates a deterministic synthetic file outside the timed section, measures three complete parser/analyzer runs, and performs a separate tracemalloc pass.

Why and benefit: separating allocation tracing avoids inflating reported throughput. Distinct-error count is configurable so memory claims can be checked. CI makes supported Python versions and installed-command checks repeatable when pushed.

Trade-off: synthetic data and warm filesystem caches may differ from production logs. Tracemalloc measures traced Python allocations, not total resident process memory. CI needs GitHub access and is not executed locally for every declared Python version. Action tags and broad dependency ranges are not an immutable lockfile.

Alternatives: timing generation with analysis obscures parser cost. Tracing during speed measurements biases results. A hard CI throughput threshold is noisy across shared hosts. A benchmark framework, profiler, or RSS monitor adds tools before a concrete regression requires them. Multiprocessing and async are deferred until measurement warrants them. The current script provides an honest baseline at low maintenance cost.

6 Technologies and concepts

Runtime libraries were reused: Pydantic validates models and serializes JSON; Typer maps annotated arguments/options into a CLI; Rich formats tables. pytest remains the test runner, and setuptools remains the packaging backend. Clean-wheel testing uses Typer 0.27.3, Rich 15.0.0, Pydantic 2.13.5, and pytest 9.1.1 on Python 3.14.4. The existing development environment uses Typer 0.27.1 and Pydantic 2.13.4. Both passed the same suite. The declared lower bounds and other Python versions were not all tested locally.

New use of standard-library facilities includes heapq for top-k selection, csv for correct quoting, tempfile and os.replace for atomic output, argparse for the standalone benchmark, statistics for median timings, tracemalloc for allocation measurement, and pathlib for path and alias handling. These add no install dependency.

The main architectural concepts are normalization adapters, lazy composition of iterables, callback-based diagnostics, separation of analysis from serialization, and atomic replacement. They solve concrete input, memory, reuse, and reliability problems without introducing a new framework.

GitHub Actions is a development service configuration, not a runtime requirement. ReportLab was used only to produce this PDF with the bundled document runtime; it is not added to pyproject and users do not need it to run Quaestio. The Markdown report is the editable source. No FastAPI, pandas, database package, ML library, or AI SDK was added. Dependency ranges were retained; a locked development environment is a future reproducibility improvement, while libraries normally allow compatible versions.

7 Problems encountered and solutions

The empty CLI caused an ImportError. Creating the Typer app and group callback restored the entry point and preserved the intended analyze subcommand rather than relying on single-command flattening. Installed-command and help tests protect that behavior.

The sample fixture was hidden by *.log. A narrow !examples/*.log exception makes only synthetic example logs eligible for tracking. MANIFEST.in also includes fixtures in source distributions. The final working tree still needs a commit before these changes appear in a fresh Git clone.

Blank lines would previously stop parsing, including the spaced sample presentation in the handoff. Stream parsing now ignores blank lines while standalone parse_line remains strict. Physically counted line locations remain correct after blanks are skipped.

Unlocated exceptions and malformed levels were hard for users to repair. Located LogParseError diagnostics, bounded skip warnings, and command-level handling of expected I/O/decoding failures provide readable errors without hiding unexpected exceptions behind catch-all handling. Validation causes remain available for Python debugging.

Exporting directly to a destination risks truncating old reports or overwriting input. Render-before-write, alias checks, same-directory temporary files, cleanup, and replace- failure tests address these cases. Resource tests confirm file handles close after normal early-stop closure and parse failure.

The first packaging check could not download setuptools inside the network sandbox. The standard pip build was rerun with authorized network access, then dependency wheels were collected. A clean virtual environment installed from those wheels without network access and ran all tests outside the project source tree. No manual bypass of package construction was used.

Broad streaming claims needed qualification. Benchmarks with 100 and 10,000 distinct errors made the cardinality cost visible. Exact counts remain exact, and the README states the O(U) bound plus per-record length and buffer costs.

8 Validation evidence and performance

Functional verification

Development suite: 58 passed. Original 11 tests remain, with fixture-path adjustments. Clean wheel suite: 58 passed using a new Python 3.14.4 virtual environment. Tests and synthetic examples were copied to a temporary directory; no src directory was copied. The loaded CLI module was verified in that environment's site-packages.

Coverage includes model construction, original text parsing, all severity counts, empty input, repeated-error ranking, top limits and ties, text/NDJSON equivalence, Unicode and file BOMs, invalid timestamps/JSON, strict and tolerant mode, warning bounds, source/severity selection, one-pass 50,000-event input, file closure, help and usage, stdin, literal Rich markup, CSV quoting, JSON stdout separation, missing/directory/ permission/decoding failures, atomic replacement, replacement failure cleanup, same-path/ symlink/hardlink protection, and module invocation outside the repository.

The wheel builds successfully with the configured setuptools backend. Installed console and module invocation succeed. git diff --check passes. The source manifest includes both fixtures and all tests. The configured remote CI has not yet run; Python 3.11-3.13 and Windows/macOS are not locally validated. No PyPI release was performed.

End-to-end benchmark results

Measured on Linux x86_64, Python 3.14.4, using the existing development dependencies. Each row uses a generated file with 10% ERROR events, three untraced runs, and a separate allocation pass. The median excludes file generation and report rendering.

Format and dataset	Median seconds	Entries per second	Peak traced bytes
Text, 10,000 entries, 100 distinct errors	0.0513	195,029	158,365
Text, 100,000 entries, 100 distinct errors	0.4999	200,056	158,365
NDJSON, 100,000 entries, 100 distinct errors	0.1330	751,621	153,282

Format and dataset	Median seconds	Entries per second	Peak traced bytes
Text, 100,000 entries, 10,000 distinct errors	0.5189	192,712	885,279

The small and large repeated-message cases showed the same traced peak, consistent with not retaining all events. Increasing distinct errors increased allocations as expected. These observations are dataset-specific, not proof of fixed memory or universal speed. NDJSON is faster in this run despite a larger file; its direct model validation path is a plausible explanation, but no detailed profile was performed to establish causality.

Reproduce these measurements with scripts/benchmark.py using --entries, --format, --distinct-errors, and --runs. The script reports Python/platform and file size along with timings; results vary with machine load, caches, record shape, and dependencies.

Ranking-only comparison

For 100,000 prebuilt distinct message counts, five timed repetitions gave mean ranking times of 0.015850 seconds for full sorting and 0.003997 seconds for heap top-five. A separate warmed tracemalloc pass recorded 8,671,088 and 408 peak traced bytes respectively, excluding the already-built dictionary. Both returned the same five tied results in first-appearance order. These narrow measurements justify the smaller ranking change; they do not establish a fourfold improvement for full log analysis.

The comparison used counts = {str(i): (i * 7919) % 997 for i in range(100000)}, and compared sorted(counts.items(), key=lambda x: x[1], reverse=True)[:5] with nlargest(5, counts.items(), key=lambda x: x[1]). Python allocation caches affect small traced peaks. Overall analyzer memory is still dominated by exact message counts.

9 Remaining work and future choices

First, commit the reviewed working tree and run the configured remote CI. Resolve cross-version or platform failures if any. Publishing a GitHub release or PyPI package is a separate next step. The local package version remains 0.1.0 to match the original initial-release scope; the changelog records the implementation milestone.

Next reliability improvements could include an immutable development dependency lock, a structured report envelope with selected/parsed/skipped counts, strict NDJSON timestamp and nonempty-string policies, and explicit resource bounds for unusually long records. Any tighter model validation should be documented as a contract change rather than silently rejecting data the old model accepted. Full RSS measurements would complement tracemalloc. Real representative datasets would strengthen performance evidence.

Logical product improvements include time-range and minimum-severity filters, source-attributed error summaries, an explicitly named ERROR-plus-CRITICAL failure metric, and parameter-aware error grouping. Each requires careful semantics, especially timezone handling and fuzzy grouping. Current timestamp fields are retained, but this release does not produce incident timelines or causal diagnoses.

Syslog and multiline stack traces require actual input examples and event-segmentation rules. Automatic detection can follow reliable explicit parsers. Schema mapping and metadata

retention should be designed together rather than bolted onto the simple model. A watch command needs rotation, truncation, partial-write, and cancellation behavior; it is not just a loop around the current file parser.

FastAPI/dashboard interfaces, SQLite history, anomaly detection, distributed tracing, and Needle are intentionally unimplemented. They fit the longer roadmap but would expand lifecycle, storage, security, statistics, or AI orchestration requirements before the first local analysis workflow is established in real use. The preserved core makes these future adapters possible without committing to their architecture now.

10 Reference material

The supplied Quaestio Master Project Handoff COMPLETE document provided the baseline architecture, project identity, original code, test checkpoint, and scope. Actual source and executed checks were used as the authority for current implementation.

Official references consulted for library behavior:

- Typer callback and command grouping: https://typer.tiangolo.com/tutorial/commands/callback/
- Typer testing and CliRunner: https://typer.tiangolo.com/tutorial/testing/
- Pydantic model serialization: https://docs.pydantic.dev/latest/concepts/serialization/
- Python heapq selection: https://docs.python.org/3/library/heapq.html
- Python os.replace: https://docs.python.org/3/library/os.html#os.replace
- GitHub setup-python action: https://github.com/actions/setup-python

These references explain the APIs; the local test and benchmark evidence establishes what this implementation actually did. The measurements and validation limits are recorded above so future changes can be compared honestly.